QAOA Portfolio Constructor: Cardinality-Constrained Stock Selection on a Quantum Simulator

Team: <team name> | Members: <name 1>, <name 2>, ... | Original project extending Track 3 (QAOA & optimisation), Qiskit Fall Fest Hackathon 2026 | Python 3.12

1. Problem

We ask a concrete finance question: **given N = 12 real stocks, which exactly k = 4 of them should an investor hold?** The objective is the classic Markowitz trade-off - maximise expected annualised return $\mu^T x$ minus a risk penalty $(\gamma/2)x^T\Sigma x$ with $\gamma = 5.0$ - subject to an exact-cardinality constraint (sum of the binary selection variables equals k). The cardinality restriction is what makes the problem genuinely hard: with continuous weights, mean-variance optimisation is a convex problem a laptop solves instantly, but once positions are all-or-nothing the search space becomes 2^N = 4096 candidate portfolios and the general problem is NP-hard. That discrete regime is precisely where combinatorial solvers - and therefore QAOA - are relevant, and it is the regime real portfolio managers face when positions must be whole lots.

Success criterion, stated before we built anything: a self-built QAOA whose best sampled portfolio reaches optimality score ≥ 0.99 at depth p ≥ 2 on real market data, with every result verified against exact enumeration of all 2^N portfolios, a fair classical baseline comparison, and honest reporting of the probability of sampling the optimal portfolio.

2. Method

- **Data.** Two years of daily adjusted closes for 12 NIFTY-50 names (Yahoo Finance), annualised to μ and Σ (252 trading days). The downloaded prices are cached in data/prices.csv so the repository also runs offline from a clean start.
- **QUBO.** $H(x) = -\mu^T x + (\gamma/2)x^T\Sigma x + P(\sum x_i - k)^2$ with penalty weight P = 10.0. Because $x_i^2 = x_i$ for binaries this is an exact quadratic form x^TQx + const, derived and coded by hand in src/qubo.py.
- **Ising mapping.** Substituting $x_i = (1-z_i)/2$ gives $H = \sum h_i Z_i + \sum J_{ij} Z_i Z_j$ + offset with $J_{ij} = Q_{ij}/4$ and $h_i = -Q_{ii}/2 - \sum_j Q_{ij}/4$ (src/ising.py). One qubit per stock; the equality constraint needs no slack qubits, so N = 12 qubits suffice.
- **Golden verification.** Before any quantum run we check that the lowest eigenvalue of the SparsePauliOp Hamiltonian plus the offset equals the exact minimum over all 4096 portfolios (tests/test_ising_mapping.py). This single test proves the whole encoding chain.
- **QAOA.** A hand-built p-layer ansatz (src/qaoa_circuit.py): Hadamard layer, cost layer $e^{-i\gamma H}$ implemented with $RZ(2\gamma h_i)$ and CX–$RZ(2\gamma J_{ij})$–CX pairs, and RX mixer layer $e^{-i\beta\sum X}$. Angles are optimised with COBYLA (300 iterations, 8 random restarts, then a Powell polish of the best point) or SPSA; the optimisation cost is the expected energy from exact statevector probabilities - a documented statistical shortcut, since no finite shots are taken during optimisation.
- **Baselines.** Exact 2^N enumeration (the reference), greedy top-k Sharpe, best-improvement local search, simulated annealing, and continuous Markowitz solved with SLSQP then rounded to k assets - all scored with the same energy ruler as QAOA.
- **Noise.** The optimised circuits are executed on AerSimulator with the calibrated FakeGuadalupeV2 noise model (16 qubits, 4096 shots) to measure how much optimal-portfolio probability and constraint feasibility survive hardware-like noise (src/noise.py).

3. Results

All numbers below come from one clean-start run (seed 42, about 5.7 minutes on a laptop) and are stored in results.json. The exact optimum selects {ICICIBANK, SBIN, BHARTIARTL, MARUTI}; the Ising-to-brute-force golden check passes to 10 decimal places.

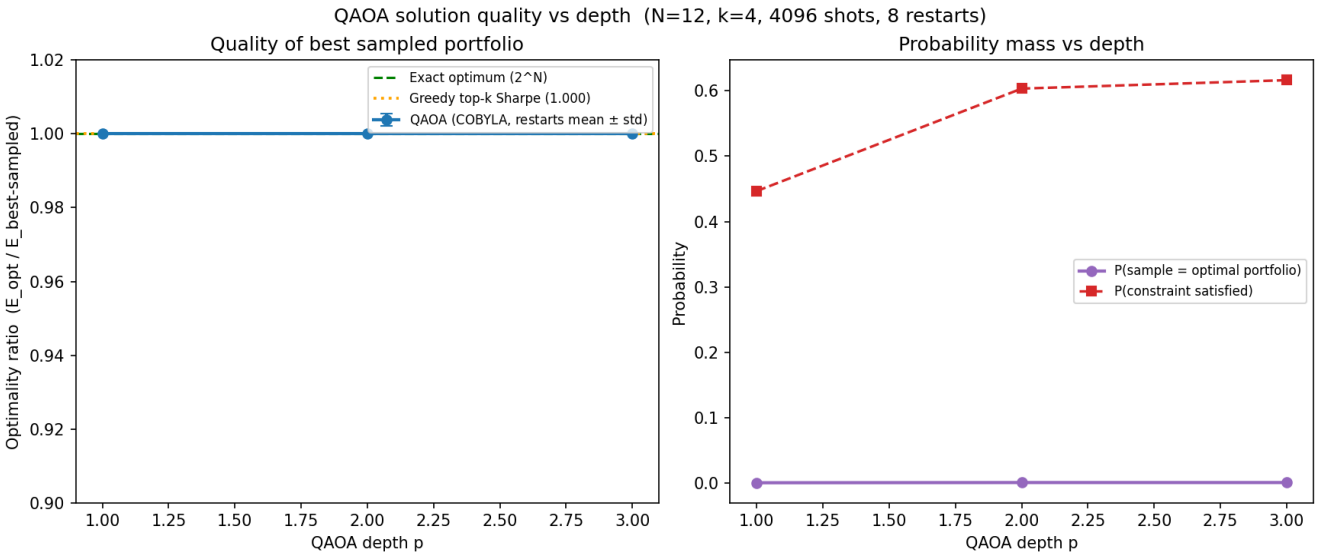


Figure 1 - Main result. Left: optimality score of the best sampled portfolio vs QAOA depth (mean ± std over 8 restarts; 1.0 = exactly optimal). Right: probability mass on the optimal portfolio and on constraint-satisfying portfolios vs depth.

Depth study. The best sampled portfolio already matches the exact optimum at p = 1 (optimality score 1.00 ± 0.00 across restarts). The probability structure is more informative: feasible-solution mass grows from 0.45 (p = 1) to 0.60 (p = 2) and 0.62 (p = 3), while P(optimal portfolio) stays near 10^{-3} - with C(12,4) = 495 feasible portfolios, low-depth QAOA spreads mass over the feasible set rather than concentrating on the single optimum. Expected energy improves from 0.132 (p = 1) to 0.075 (p = 2) against an exact optimum of –0.002 in scaled units, showing the angle search - not the encoding - is the bottleneck.

Optimisers. COBYLA+Powell reaches expected energy 0.075 vs 0.138 for SPSA (same budget class) - neither converges to the exact optimum, and reporting this gap is more useful than hiding it.

Finance view. Sweeping the risk-aversion parameter γ traces the cardinality-constrained efficient frontier (Figure 2): QAOA's best sampled portfolios lie exactly on the frontier computed by brute force at every γ, with annualised returns of 9.8-25.7% for volatilities of 12.5-15.0%. Greedy top-k Sharpe lands on the aggressive end and is dominated once risk aversion grows.

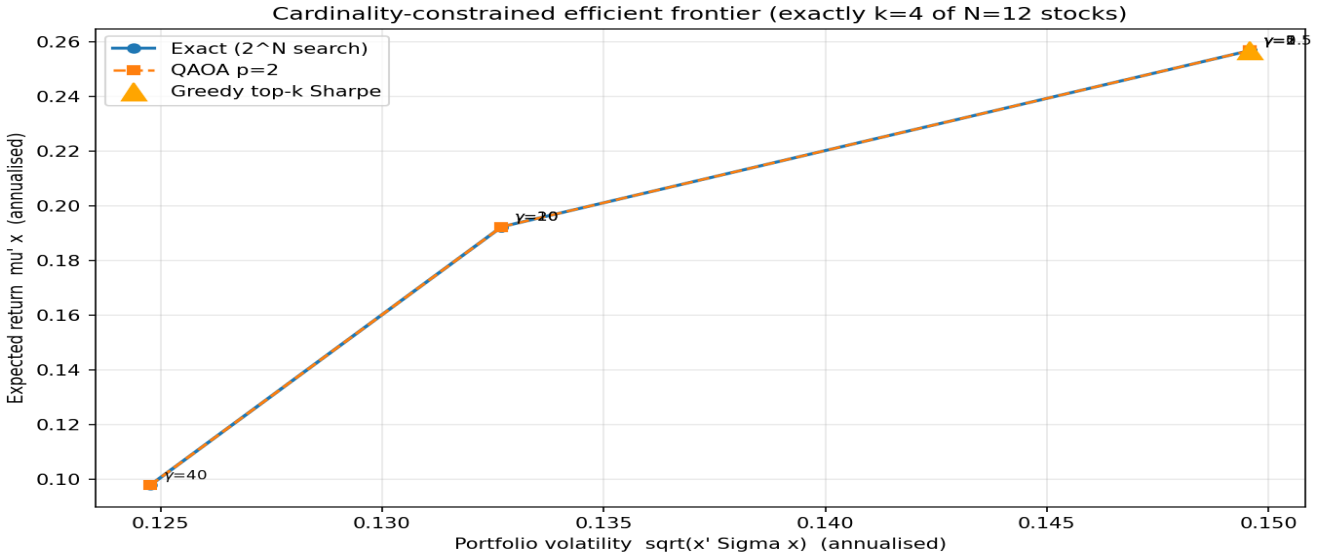


Figure 2 - Cardinality-constrained efficient frontier (exactly k = 4 of 12 stocks): exact 2^N search vs QAOA (p = 2) vs greedy top-k Sharpe.

Constraint and robustness. Sweeping the penalty weight P from 0.5 to 50 leaves the best-sampled solution optimal throughout and the feasible rate between 0.44 and 0.46 at p = 1, so the chosen P = 10 sits comfortably inside the stable region. Under the FakeGuadalupeV2 noise model with the same optimised angles, feasible-solution mass drops from 0.45/0.60/0.62 (ideal, p = 1/2/3) to 0.29/0.25/0.09: deeper circuits accumulate far more noise, so on hardware-like conditions p = 1 is the most noise-robust operating point - the opposite of the ideal-depth trend, which is exactly the trade-off a hardware team must manage.

Classical baselines (same ruler). Exact enumeration, greedy, local search and annealing all reach optimality score 1.0 in milliseconds; continuous Markowitz + rounding scores 0.9995. We do not claim a quantum advantage: at N = 12 classical methods solve this instance exactly, and the honest value of the quantum pipeline here is verification, methodology and the characterisation above - not speed.

4. Honest limits

- **Scale.** N = 12 is chosen so that 2^N enumeration stays a valid referee. Around N ≈ 25 enumeration dies; we did not test that regime, so no scaling claim is made.
- **Convergence.** Expected energy plateaus at ~0.075 vs an exact –0.002: better initial angles, warm starts or deeper p with mitigation might close the gap; none of these were tested.
- **Statistics.** Angle optimisation uses exact statevector probabilities (no shot noise); shot-level effects enter only through the noise study. Reported probabilities use 4096 samples from the ideal distribution.
- **Finance simplifications.** Mean-variance only: no transaction costs, taxes, lot sizes or sector limits; μ and Σ assumed stationary over the 2-year window; no out-of-sample evaluation was run.
- **Encoding trade-off.** The quadratic penalty avoids slack qubits but reshapes the landscape; a slack-variable encoding was not compared.

5. AI-use note

AI assistants were used for scaffolding code, drafting derivations and debugging. **One thing the AI got wrong:** the golden test initially compared the Ising Hamiltonian's lowest eigenvalue directly with the brute-force minimum, omitting the constant offset that our build_hamiltonian deliberately excludes (QAOA is invariant to global energy shifts); the n = 4 check reported an eigenvalue of –1.24 against a true minimum of 1.85. **How we caught it:** tests/test_ising_mapping.py compares exact diagonalisation against enumeration of all 2^N energies, and a hand-checked n = 2 case located the discrepancy in the constant term; the fix adds the offset before comparing. Every team member reviewed the QUBO derivation, the Ising substitution, the RZ/CX cost-layer decomposition and the figures.

References. Farhi, Goldstone & Gutmann (2014), A Quantum Approximate Optimization Algorithm; Markowitz (1952), Portfolio Selection; Qiskit and Aer documentation; the organisers' submission template and guidelines.